UNIVERSITETI I PRISHTINËS
FAKULTETI I SHKENCAVE MATEMATIKE-NATYRORE
PROGRAMI SHKENCA KOMPJUTERIKE



ANALIZA E TË DHËNAVE

IMDb Movies Dataset

K-Means dhe Apriori

Punuan:

Elvira Maloku
Natyrë Halabaku
Yllka Ejupi

Profesor: Eliot Bytyci

Asistent: Shkemb Abdullahu

Hyrje

Për fazën e dytë të projektit kemi përdorë prap datasetin e filmave nga IMDb qe e kemi marrë në Kaggle, ku atributet përshkruajnë karakteristikat e filmave. Atributet e datasetit janë viti i publikimit, certifikata, kohëzgjatja, zhanri, rating, Metascore, regjisori, numri i votave dhe numri i reviews.

Në pjesën e parë të projektit jemi marrë me klasifikim, ku qëllimi ishte që filmat me u nda në tri kategori: filma të dobët, mesatar dhe filma të mirë. E në pjesën e dytë fokusi është te grupimi (clusterimi) i te dhenave dhe gjetja e rregullave me association rule mining.

Kemi përdorë dy algoritme kryesore:

- **K-Means**, për grupimin e filmave në klasterë.
- **Apriori**, për gjetjen e rregullave të shoqërimit ndërmjet karakteristikave të filmave.

Qëllimi kryesor i kësaj analize është të shihet se si grupohen filmat në bazë të karakteristikave të tyre dhe cilat lidhje ekzistojnë mes atributeve si zhanri, certifikata, regjisori, Metascore, votat, viti dhe kategoria e rating, që shfaqen më së shpeshti.

```
- DATA SUMMARY -
Total columns: 9
Numeric columns: 4
Categorical columns: 5

Numeric column names: ['Year', 'Duration (min)', 'Rating', 'Metascore']
Categorical column names: ['Certificate', 'Genre', 'Director', 'Votes', 'Review Count']

Missing values per column:
Certificate       2630
Metascore         2445
Votes              404
Rating             404
Duration (min)     336
Year               150
Genre                7
Director             5
Review Count         1
dtype: int64
```

Figura 1: K-Means pas preprocessing

Po shohim se sipas analizës fillestare dataseti përmban kolona numerike dhe kategorike. Disa prej tyre kanë vlera që mungojnë, ndërsa disa kolona numerike si `Votes` dhe `Review Count` përmbajnë presje, kështu që duhet pastruar.

Përzgjedhja e kolonave

Dataseti i marrë nga Kaggle kishte shumë kolona, ku disa prej tyre përmbajnë të dhena që nuk kontribuojnë në analizen që jemi duke e bërë dhe janë të përshtatshme për algoritmet e përdorura, sepse disa kanë shumë missing values.

Prandaj, kemi zgjedhur këto atribute:

- Year
- Certificate
- Duration
- Genre
- Rating
- Metascore
- Director
- Votes
- Review Count

Feature Engineering

Feature engineering është një nga hapat më të rëndësishëm në analizën e të dhënave, sepse në këtë hap bëhet përgatitja e të dhenave për analize dhe ato bëhen më të përshtatshme për algoritmet qe jemi duke i përdorë.

Fillimisht, kolonat `Votes` dhe `Review Count` janë kthyer në numerike. Kjo bëhet sepse këto kolona mund të kenë presje dhe algoritmet nuk mund t'i trajtojnë si tekst.

```
df["Votes"] = pd.to_numeric(df["Votes"].astype(str).str.replace(",","), errors="coerce")
df["Review Count"] = pd.to_numeric(df["Review Count"].astype(str).str.replace(",",""), errors="coerce")
```

Listing 1: Pastrimi i kolonave numerike

Transformimi tjetër është bërë mbi atributin `Genre`. Në dataset, një film mund të ketë më shumë se një zhanër, për shembull *Action, Adventure, Sci-Fi*. Për të evituar the Curse of Dimensionality, është marrë vetëm zhanri i parë, me c'rast thjeshtohet analiza.

```
df["Genre"] = df["Genre"].astype(str).str.split(",").str[0].str.
    strip()
```

Listing 2: Marrja e zhanrit të parë

Nga pjesa e parë, atributin `Rating` e kemi kthyer në kategori. E mira e kësaj është që tani nuk punojmë më vetëm me vlera numerike, por filmat ndahen në tri kategori: të dobët, mesatar dhe të mirë. Përveq kesaj, në gjetjen e association rules te Apriori ku kërkohen lidhje mes kategorive të ndryshme, kjo ndihmon shumë në rezultat.

```
def rating_cat(r):
    if pd.isna(r): return np.nan
    if r < 5.5: return "Bad"
    if r <= 7.0: return "Average"
    return "Good"

df["RatingCategory"] = df["Rating"].apply(rating_cat)
df = df.dropna(subset=["RatingCategory"])

num_cols = ["Year","Duration (min)","Metascore","Votes","Review
    Count"]
cat_cols = ["Certificate","Genre","Director"]
```

Listing 3: Krijimi i kategorisë së rating-ut

Para Preprocesimi

Në këtë pjesë, para se të aplikojmë algoritmin K-Means, kemi bërë një përpunim minimal të të dhënave ashtu që ato të jenë më të përshtatshme për të. Pra, ende nuk janë trajtuar outliers, shkallëzuar të dhënat apo transformuar me logaritëm. Cka kemi bërë është përdorimi i kolonave numerike: `Metascore`, `Votes`, `Review Count` e jo atyre kategorike, për arsye se ato kanë kuptim të drejtëpërdrejtë për K-Means.

```
df_before = df.copy()

cluster_cols = ["Metascore", "Votes", "Review Count"]

for c in cluster_cols:
    df_before[c] = df_before[c].fillna(df_before[c].median())

X_before = df_before[cluster_cols]
```

Listing 4: Përgatitja e të dhënave para preprocessing-ut

Duhet të ceket që rezultati në këtë pjesë mund të ndikohet nga vlerat ekstreme dhe nga kolonat me shpërndarje shumë të madhe.

K-Means para preprocessing-ut

K-Means është algoritëm i unsupervised learning që përdoret për ndarjen e të dhënave në grupe. Këto grupe quhen klastere. Në analizen tonë kemi përdorë `n_clusters=3`, që do të thotë se filmat ndahen në tri klastere. Më poshtë është paraqitur vizulalizimi i klasterit përmes PCA.

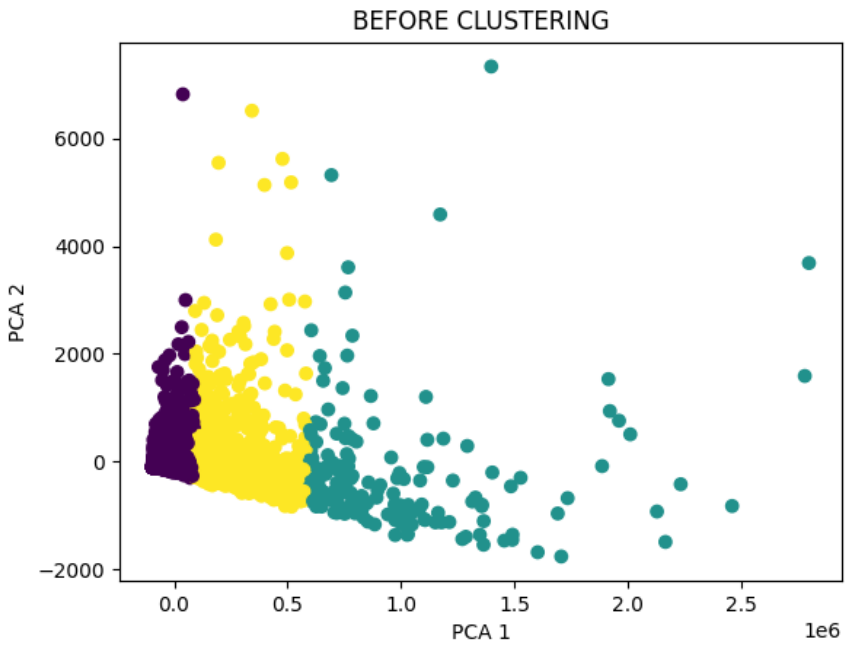


Figura 2: K-Means pas preprocessing-ut

Gjithashtu përmes Silhouette Score dhe Davies-Bouldin Score vlerësohet ndarja e klasterëve. Silhouette Score tregon sa mirë janë ndarë klasterët, ndërsa Davies-Bouldin Score tregon sa të ngjashëm janë klasterët mes vete. Te Silhouette vlera më e madhe është më e mirë, ndërsa te Davies-Bouldin vlera më e vogël është më e mirë.

Preprocesimi

Preprocesimi është faza më e rëndësishme në analizë, sepse në këtë fazë bëhet pastrimi dhe përgatitja më e mirë e të dhënave. Kjo fazë është e nevojshme për përfitimin e rezultateve reale, pa ndikimin e formave dhe elementeve të panevojshme.

Para se me fillu me përpunimin e të dhënave është krijuar një kopje e datasetit për pjesën pas preprocessing.

Çka bëmë fillimisht është trajtimi i vlerave që mungojnë për kolonat `Year` dhe `Duration (min)`, duke i plotësuar me medianë. Kjo sepse këto përdoren më vonë në pjesën e Apriori-t për krijimin e kategorive.

Ndërsa për kolonat kategorike vlerat janë plotësuar me vlerën `Unknown`.

Tash te kolona `Director` është bërë reduktimi i kategorive, sepse kjo kolonë ka shumë vlera unike gjë që e fryn datasetin dhe e komplikon analizen, prandaj i kemi mbajtur vetëm 20 regjisorët më të shpeshtë, ndërsa të tjerët janë vendosur në kategorinë `Other`.

```
top_directors = df_after["Director"].value_counts().head(20).index
df_after["Director"] = df_after["Director"].apply(
    lambda x: x if x in top_directors else "Other"
)
```

Listing 5: Reduktimi i kategorive te Director

Tjetër problem që ndikon në clustering janë outliers. K-Means është algoritëm që bazohet në distancë, prandaj ndikohet shumë nga outliers. Për trajtimin e tyre kemi vendos me përdor tekniken `quantile clipping`, e cila kufizon vlerat shumë të vogla dhe shumë të mëdha në intervalin 5% - 95%. Këtë e kemi aplikuar vetëm në kolonat që përdoren në klasterim: `Metascore`, `Votes` dhe `Review Count`.

```
for c in cluster_cols:
    low, high = df_after[c].quantile([0.05, 0.95])
    df_after[c] = df_after[c].clip(low, high)
```

Listing 6: Trajtimi i outliers

Pas trajtimit të outliers, vlerat që mungojnë në kolonat e klasterimit plotësohen me medianë.

Dhe në fund në kolonat `Votes` dhe `Review Count` shpërndarja është shumë e madhe, prandaj është përdorur transformimi logaritmik.

```
df_after["Votes"] = np.log1p(df_after["Votes"])
df_after["Review Count"] = np.log1p(df_after["Review Count"])
```

Listing 7: Transformimi logaritmik

K-Means pas preprocessing-ut

Pas preprocessing, për K-Means janë përdorur kolonat `Metascore`, `Votes` dhe `Review Count`, të cilat janë të njëjtat që i kemi përdor edhe para preprocessing, veçse

tani të dhënat janë të pastra, të kufizuara nga outliers, janë transformuar dhe shkallëzuar.

Për shkak të outliers, menduam që më e përshtatshme për shkallëzim është të përdoret `RobustScaler`, pasi që është më rezistent ndaj outliers.

```
scaler = RobustScaler()
X_after_scaled = scaler.fit_transform(X_after)

kmeans_a = KMeans(n_clusters=3, random_state=RANDOM_STATE, n_init
    =20)
clusters_a = kmeans_a.fit_predict(X_after_scaled)

sil_a = silhouette_score(X_after_scaled, clusters_a)
db_a = davies_bouldin_score(X_after_scaled, clusters_a)
```

Listing 8: K-Means pas preprocessing-ut

Edhe këtu e kemi shtuar vizualizimin në dy dimensione pas preprocessing njësoj me PCA.

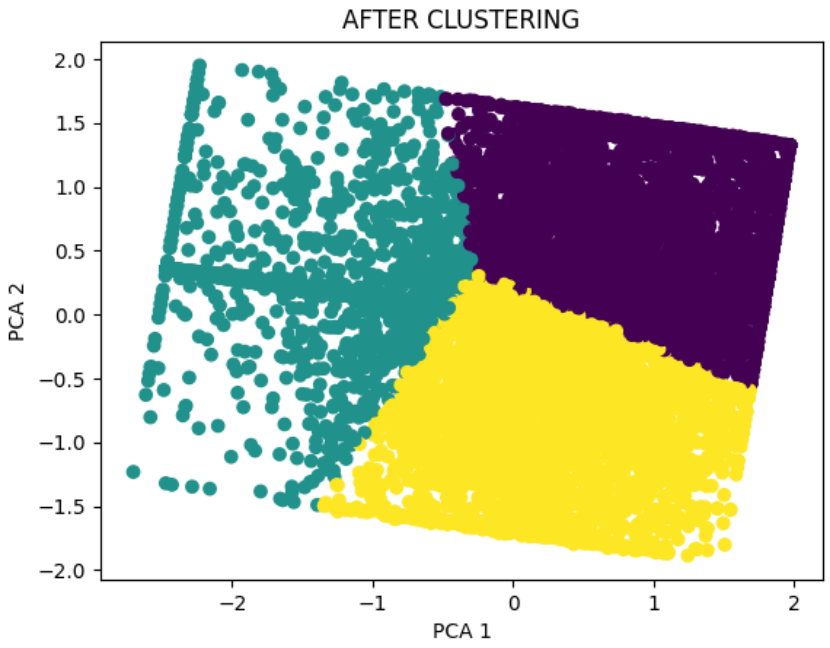


Figura 3: K-Means pas preprocessing-ut

Nëse i karahsojmë figurat shohim se si është bërë klusterimi i të dhënave para edhe pas preprocessing, ku ngjyrat përfaqesojne 3 klasteret. Secila pikë = një film.

Rezultatet e K-Means

Çka mund të nxjerrim nga rezultatet?

Ashtu si e cekem më lart, për vlerësimin e ndarjes së klasterëve janë përdorë metrikat Silhouette Score dhe Davies-Bouldin Score.

```
- BEFORE CLUSTERING -
Columns used: ['Metascore', 'Votes', 'Review Count']
Silhouette: 0.7642005586108898
Davies-Bouldin: 0.5367520693573732
Cluster sizes:
0    8282
2    1126
1     188
Name: count, dtype: int64

- AFTER CLUSTERING -
Columns used: ['Metascore', 'Votes', 'Review Count']
Silhouette: 0.36580207958964706
Davies-Bouldin: 0.9673717320263099
Cluster sizes:
2    3885
0    3237
1    2474
Name: count, dtype: int64
```

Figura 4: Silhuette Score dhe Davies-Bouldin Score

Nga ky krahasim shohim se Silhuette Score ka qenë më e lartë para preprocessing ndërsa pas më e ulët gjë qe nuk është optimale. Mirëpo rezultati para është ndikuar shumë nga shpërndarja jo e rregullt e të dhënave, për shembull në rastin e Votes Count, kemi film që ka miliona vota dhe film që ka 2k vota, e kjo bënë që të krijohen ndarje artificiale mes grupeve. Kjo mund t'i rrisë këto metrika edhe pse clustering nuk është domosdoshmërisht më realist. Pas preprocessing aplikimi i transformimeve bënë që klasterat të jenë më real, por edhe pikat të jenë më afer njëra tjetrës në hapësirë. Si rezultat, metrikat dolen më të dobëta, por klasterimi pas preprocesimit konsiderohet më i besueshëm.

Apriori - Association Rule Mining

Për gjetjen e rregullave ose lidhjeve mes elementeve në dataset kemi përdorë algoritmin Apriori. Qëllimi është të gjejmë se cilat elemente paraqiten më shpesh së bashku.

Në rastin tonë, e përdorem Apriorin për të analizuar lidhjet ndërmjet zhanrit, certifikatës, regjisorit, rating, votave, Metascore, kohëzgjatjes dhe vitit të publikimit të filmave.

Për ta realizuar këtë, fillimisht është dashur që disa kolona numerike të kthehen në kategori. Kategoritë janë krijuar ashtu që të kenë kuptim për secilen kolonë të përzgjedhur.

```
df_rules = df_after.copy()

df_rules["VotesCat"] = pd.cut(df_rules["Votes"], 3, labels=["Low",
    "Medium","High"])
df_rules["MetaCat"] = pd.cut(df_rules["Metascore"], 3, labels=["
    Low","Medium","High"])
df_rules["DurCat"] = pd.cut(df_rules["Duration (min)"], 3, labels
    =["Short","Medium","Long"])
df_rules["YearCat"] = pd.cut(
    df_rules["Year"],
    bins=[0, 2000, 2015, 3000],
    labels=["Before_2000", "2000_2015", "2015_plus"]
)
```

Listing 9: Kategorizimi i kolonave për Apriori

Me pas janë zgjedhur kolonat që përdoren për algoritmin e Apriorit: Genre, Certificate, Director, RatingCategory, VotesCat, MetaCat, DurCat, YearCat. Dhe dataseti është kthyer në one-hot encoding për përpunim me të dhëna kategorike në boolean.

Pastaj është aplikuar algoritmi Apriori me minsup = 0.05, që do të thotë se një itemset duhet të paraqitet të paktën në 5% të datasetit. Vlera 0.05 është zgjedhur sepse dataseti ka shumë kombinime të mundshme të elementeve, dhe nëse suporti do të ishte më i vogël atëherë do të gjeneroheshin rregulla të rastësishme dhe të panevojshme. Në anën tjetër, nëse suporti do të ishte më i madh, do të humbnim shumë rregulla interesante që lidhen me kategori më specifike.

```
freq = apriori(encoded, min_support=0.05, use_colnames=True)
```

Listing 10: Aplikimi i Apriori

Association Rules

Nga frequent itemsets janë gjeneruar association rules (rregullat e shoqerimit). Këto rregulla vlerësohen me tri metrika kryesore:

- **Support** – tregon sa shpesh paraqitet rregulli në dataset.

- **Confidence** – tregon probabilitetin që consequent të ndodhë kur antecedent është i pranishëm.
- **Lift** – tregon sa e fortë është lidhja krahasuar me rastësinë.

  - per lift > 1, lidhja është pozitive

  - per lift $= 1$, lidhja është e rastësishme

  - per lift < 1, lidhja është negative

Filtrimi i rregullave

Pas krijimit të rregullave, është me rëndësi që disa rregulla të filtrohen për shkak se jo të gjitha rregullat e gjeneruara janë të dobishme ose interesante. Disa mund të jenë shumë të përgjithshme dhe triviale ose të bazuara në vlera Unknown. Për këtë arsye, janë larguar rregullat që përmbajnë `Unknown` dhe disa rregulla shumë të dukshme.

```
def is_noisy(rule):
    a = str(rule["antecedents"])
    c = str(rule["consequents"])

    if "Unknown" in a or "Unknown" in c:
        return True

    if "RatingCategory_Good" in a and "MetaCat_High" in c:
        return True

    return False

rules_clean = rules[~rules.apply(is_noisy, axis=1)]
```

Listing 11: Filtrimi i rregullave jo interesante

Pas këtij filtrimi janë mbajtur vetëm rregullat më interesante, duke përdorur kushtet për support, confidence dhe lift.

```
interesting = rules_clean[
    (rules_clean["lift"] >= 1.2) &
    (rules_clean["confidence"] >= 0.55) &
    (rules_clean["support"] >= 0.03)
]
```

Listing 12: Zgjedhja e rregullave më interesante

E në fund i kemi zgjedhur ato rregulla që kanë më shumë kuptim për interpretim, pra ato që përmbajnë `DurCat`, `Certificate`, `Director`, `Genre` ose `YearCat` në pjesën e majtë (antecedent).

```python
interesting_story = interesting[
    (
        interesting["antecedents"].str.contains("DurCat") |
        interesting["antecedents"].str.contains("Certificate") |
        interesting["antecedents"].str.contains("Director") |
        interesting["antecedents"].str.contains("Genre") |
        interesting["antecedents"].str.contains("YearCat")
    )
]
```

Listing 13: Zgjedhja e rregullave më interpretuese

Me anë të një filtri e kemi rregulluar ashtu që të paraqiten vetëm rregullat që kanë 2 e më shumë items në pjesën e majtë për rezultate më interesante.

Rezultatet e Apriori

Me anë të këtyre rezultateve, algoritmi Apriori tregon se si atributet e filmave lidhen mes veti. Rëndesia e rregullave caktohet nga metrika `lift`, pra, sa më e lartë, aq më e rëndesishme.

- TOP INTERESTING ASSOCIATION RULES -

	antecedents	consequents	support	confidence	lift
1236	RatingCategory_Bad, MetaCat_Medium, Director_O...	VotesCat_Low, DurCat_Short	0.056794	0.675341	3.949159
1232	RatingCategory_Bad, MetaCat_Medium, DurCat_Sho...	VotesCat_Low	0.056794	0.675341	3.946754
806	RatingCategory_Bad, MetaCat_Medium, Director_O...	VotesCat_Low	0.056794	0.675341	3.946754
990	RatingCategory_Bad, MetaCat_Medium, DurCat_Short	VotesCat_Low	0.057211	0.669512	3.912691
1237	RatingCategory_Bad, MetaCat_Medium, DurCat_Short	VotesCat_Low, Director_Other	0.056794	0.664634	3.905590
1096	RatingCategory_Average, Certificate_UA	VotesCat_High, DurCat_Short, Director_Other	0.050542	0.728228	1.861502
1095	RatingCategory_Average, DurCat_Short, Certific...	VotesCat_High, Director_Other	0.050542	0.728228	1.861006
643	RatingCategory_Average, Certificate_UA	VotesCat_High, Director_Other	0.050542	0.728228	1.861006
226	DurCat_Short, Certificate_UA	VotesCat_High	0.097228	0.782062	1.847985
659	RatingCategory_Average, Certificate_UA	VotesCat_High, DurCat_Short	0.054085	0.779279	1.841863
223	RatingCategory_Average, Certificate_UA	VotesCat_High	0.054085	0.779279	1.841409
658	RatingCategory_Average, DurCat_Short, Certific...	VotesCat_High	0.054085	0.779279	1.841409
1094	RatingCategory_Average, Director_Other, Certif...	VotesCat_High, DurCat_Short	0.050542	0.774760	1.831182
1092	RatingCategory_Average, DurCat_Short, Director...	VotesCat_High	0.050542	0.774760	1.830732
642	RatingCategory_Average, Director_Other, Certif...	VotesCat_High	0.050542	0.774760	1.830732
651	DurCat_Short, Certificate_UA	VotesCat_High, Director_Other	0.088995	0.715842	1.829354
652	Director_Other, Certificate_UA	VotesCat_High, DurCat_Short	0.088995	0.772851	1.826669
217	Director_Other, Certificate_UA	VotesCat_High	0.088995	0.772851	1.826219
649	DurCat_Short, Director_Other, Certificate_UA	VotesCat_High	0.088995	0.772851	1.826219
1301	MetaCat_High, YearCat_2015_plus, RatingCategor...	VotesCat_High, DurCat_Short	0.050750	0.726866	1.717981

Figura 5: Top Association Rules

Këtu paraqiten rregullat që shoqërohen shpesh bashkë. Për shembull:

Rregulla 1096: RatingCategory_Average, Certificate_UA -> VotesCat_High, DurCat_Short, Director_Other

Kjo rregull tregon se filmat me rating mesatar dhe certifikatë UA kanë tendencë të kenë numër të lartë votash, të kenë kohëzgjatje të shkurtë dhe të kenë regjisorë më

pak të shpeshtë në dataset.

Rregulla 87: MetaCat_High, YearCat_2015_plus, RatingCategory_Good-> VotesCat_High, DurCat_Short

Ose rregulla që kanë lift të madh sepse janë triviale:

Rregulla 1236: RatingCategory_Bad, MetaCat_Medium, Director_Other -> VotesCat_Low, DurCat_Short

Kjo rregull tregon se filmat e dobët, me Metascore mesatar dhe me regjisorë më pak të shpeshtë në dataset, zakonisht kanë pak vota dhe janë më të shkurtër.

Përfundimi

Në këtë projekt u analizua dataseti i filmave IMDb duke përdorur algoritmet K-Means dhe Apriori. Fillimisht është përpunuar në masë të vogël dataseti ashtu që të jetë i përshtatshëm për K-Means i cili bazohet në distanca mes pikave. Qëllimi ishte të krahasohen rezultatet para edhe pas preprocessing, për të parë se sa ndikon preprocesimi në klasterimin e të dhënave. Për ndarjen e klastereve me K-Means u përdorën të njëjtat kolona në dy fazat `Metascore`, `Votes` dhe `Review Count`. Pastaj analizuam rezultatet dhe u vërejt dukshëm ndikimi i përpunimit/transformimeve të të dhënave në klasterimin e tyre. Para preprocessing, klasterimi mund të ndikohet nga outliers dhe nga kolonat me shkallë të ndryshme. Pas preprocessing, të dhënat bëhen më të pastra dhe më të përshtatshme për analizë. Në anën tjetër algoritmin Apriori e përdorëm për të gjetur rregulla të shoqërimit mes atributeve të filmave. Por për mos me dalë rregulla të rastësishme apo triviale, i kemi filtruar ashtu që të zbulojmë rregulla më interesante. Dhe nga rezultatet pamë që ekzistojnë disa kombinime të zhanreve, ratings dhe metascore që lidhen shpesh me filma të mirë, ose rregulla ku disa regjisorë ose kategori filmash shfaqen shpesh me high ratings. Si përfundim, përdorimi i dy algoritmeve K-Means dhe Apriori ndihmuan për të kuptuar më mirë datasetin dhe lidhjet mes atributeve të filmave.